Lab Assignment 6: Creating and Connecting to Databases

DS 6001

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

Please note: you will not be able to use Rivanna for this lab as Rivanna is not set up to work with Docker or with Databases. Problem 1 will guide you through the steps to get databases running on your local system. This can be tricky, so if you have questions or would like some help, please let me know.

Problem 0

Import the following packages:

In [74]:
```
import numpy as np
import pandas as pd
import requests
import json
import sqlite3
import psycopg
import mysql.connector
from sqlalchemy import create_engine
import pymongo
from bson.json_util import loads, dumps
import dotenv
import os
```

Problem 1

Many database systems run as external software on a computer. Some of the software is commercial (Oracle, Microsoft SQL Server), with limited free use and expensive enterprise level licenses. Other database software is entirely free and open source (MySQL, Postgres, SQlite, MongoDB). Regardless of whether the software is free or not, databases are notoriously difficult to install and start using. More than other kinds of software, database systems seem to run into many errors that are specific to the

operating system of your computer. We used to ask students to install Postgres, MySQL, and Mongo, and it was a nightmare because everyone hit some sort of roadblock and those roadblocks were different for every student.

That's just how databases seem to be! Not just for students, but for everyone!

A growing standard practice in industry is to use **containers** to deploy database systems. Please read the [discussion of containers in the course textbook], (https://jkropko.github.io/surfing-the-data-pipeline/docker.html) if you haven't yet done so to familiarize yourself with the purpose and uses of containers and Docker.

This lab will guide you through the installation steps for the software you need to run database systems on your computer (problem 1), build a relational schema (problem 2), document those databases (problem 3), and connect to them through Python with (fingers crossed) as few problems as possible (problem 4 for relational databases, problem 5 for a NoSQL database).

With the exception of SQLite (which operates as a Python package), database systems run as external software that must be installed and run on your computer. To make the installation steps easier, you will need some configuration files that I wrote and saved in a GitHub repository. Open your terminal and then type

```
git clone https://github.com/jkropko/ds6001databases
```

If this command works, it will create a new directory within your current folder called "ds6001databases".

- Check that this folder exists and contains the following files: LICENSE, README.md, compose.yaml, and requirements.txt. (Don't worry if there are a couple extra files.)
- To make things easier, save the notebook file you will be using for your Lab 6 work inside the "ds6001databases" folder

Part a

After cloning the ds6001databases repository, open and examine the file called compose.yaml. Then write answers to the following questions based on your reading of the section on Docker compose files in the textbook:

1. How many containers are being launched when we execute this compose.yaml file?

Three containers, MySQL, PostgreSQL and MongoDB

2. What does each of the containers do?

Each one runs a different database system, relational databases (MySQL and PostgreSQL) and NoSQL (MongoDB)

3. Docker is building these containers from Docker image files that it downloads from a website. On what website are these Docker images stored? (You can provide the general name of this website, no need for the specific URLs)

Docker Hub

[4 points]

Part b

If you haven't yet done so, install Docker Desktop on your computer. Go to https://www.docker.com/products/docker-desktop/ and click on the Download button, making sure the operating system listed matches the operating system of your computer (for Apple users, make sure you get the correct version for your computer's processing chip. Click the Apple icon in the top-left corner of your screen, select "About This Mac", and see whether Intel or Apple M1 is listed under Chip).

Once Docker Desktop is installed, find the Docker Desktop program on your computer and run it.

To confirm that Docker Desktop is running, open a terminal and type `docker version`. Copy-and-paste what you see on the screen into your notebook.

If your terminal hangs without displaying anything, that likely means that Docker Desktop is not running or not properly installed. Double check that the Docker Desktop is running. If your terminal is hanging, you can press Control + C to regain access to the command prompt.

[4 points]

PS C:\Users\carlo> docker version Client: Version: 29.5.3 API version: 1.54 Go version: go1.26.4 Git commit: d1c06ef Built: Wed Jun 3 18:03:06 2026 OS/Arch: windows/amd64 Context: desktop-linux

Server: Docker Desktop 4.78.0 (229452) Engine: Version: 29.5.3 API version: 1.54 (minimum version 1.40) Go version: go1.26.4 Git commit: 285b471 Built: Wed Jun 3 17:59:56 2026 OS/Arch: linux/amd64 Experimental: false containerd: Version: v2.2.4 GitCommit: 193637f7ee8ae5f5aa5248f49e7baa3e6164966e runc: Version: 1.3.5 GitCommit: v1.3.5-0-g488fc13e docker-init: Version: 0.19.0 GitCommit: de40ad0 PS C:\Users\carlo>

Part c

Inside your "ds6001databases" folder, create a file named `.env`. Inside the .env file you need to choose passwords for the MySQL, PostgreSQL, and MongoDB databases, so type

```
MYSQL_ROOT_PASSWORD=password1
POSTGRES_PASSWORD=password2
MONGO_INITDB_ROOT_PASSWORD=password3
MONGO_INITDB_ROOT_USERNAME=mongo
mongo_init_db = mongodb
MYSQL_DATABASE=mysql
```

Change the passwords on the first three lines to whatever you want, but DON'T USE THE @ SYMBOL as that will cause problems. Leave the fourth, fifth, and sixth lines alone, as well as the names of each environmental variable.

Then run this line of Python code:

```
dotenv.load_dotenv()
```

If the .env file has successfully been created and saved in the correct location, the output of this code will be `True`. If `False`, double check the name and location of your file.

[4 points]

In [75]:
```
dotenv.load_dotenv()
```

Out[75]:
```
True
```

Part d

In the terminal, make sure you are in the "ds6001databases" folder (you can check by typing `pwd`. If not, then use `cd` to navigate to the "ds6001databases" folder). Then type

```
docker compose up
```

This command launches all of the containers for running each of the database systems. If successful, you will see a long stream of output with messages that begin `ds6001databases-postgres-1`, `ds6001databases-mysql-1`, and `ds6001databases-mongo-1`. Copy and paste below the first six lines of this output. (Apologies for all the scrolling you might have to do!)

If you receive an error message or if the terminal hangs without displaying output, you will likely need to double-check that parts a, b, and c can still be completed successfully, and that you are working within the "ds6001databases" folder via the terminal.

[4 points]

\lab_assignment6\ds6001databases> docker compose logs --no-color | Select-Object -First 6 mysql-1 | 2026-10-19 00:24:08+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 26.7.0-1.el9started. mysql-1 | 2026-10-19 00:24:08+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql' mysql-1 | 2026-10-19 00:24:08+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 26.7.0-1.el9started. mysql-1 | 2026-10-19 00:24:09+00:00 [Note] [Entrypoint]: Initializing database files mysql-1 | 2026-10-19T00:24:09.111486Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start. postgres-1 | The files belonging to this database system will be owned by user "postgres".

Part e

To confirm that the databases are running on your system, run the following Python code.

```
# 1. Load needed environment variables
dotenv.load_dotenv()
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD')
MONGO_INITDB_ROOT_USERNAME =
os.getenv('MONGO_INITDB_ROOT_USERNAME')
MONGO_INITDB_ROOT_PASSWORD =
os.getenv('MONGO_INITDB_ROOT_PASSWORD')
MYSQL_ROOT_PASSWORD = os.getenv('MYSQL_ROOT_PASSWORD')

# 2. Test MySQL
dbserver = mysql.connector.connect(
    user='root',
    password=MYSQL_ROOT_PASSWORD,
    host='localhost',
    port='3306',
    #auth_plugin='mysql_native_password'
)

# 3. Test PostgreSQL
dbserver = psycopg.connect(
    user='postgres',
    password=POSTGRES_PASSWORD,
    host='localhost',
    port = '5432'
)
dbserver.autocommit = True

# 4. Test MongoDB
myclient =
pymongo.MongoClient(f"mongodb://{MONGO_INITDB_ROOT_USERNAME}:
{MONGO_INITDB_ROOT_PASSWORD}@localhost:27017/")
myclient.list_databases()
```

If this code runs without error, you will see output that looks something like

```
<pymongo.synchronous.command_cursor.CommandCursor at
0x177d8b380>
```

and you are all set for working with these databases using Python.

If you do see an error, then you will need to troubleshoot. A few things to try:

- Double check that your work for parts a, b, c, and d can still run successfully.
- Read the error to determine which of the databases is causing the error, to narrow the problem down. Comment-out or delete this part of the code to see if the other databases are working.
- Check the output in the terminal where you issued the `docker compose up` command. Scroll up and down, and look for any lines that write that a container has exited.
- Try a reset of the containers by pressing Control+C in the terminal displaying Docker output, then `docker compose down`, then `docker compose up` again. Then try running the Python code again.
- Sometimes (especially if you've changed the database passwords at some point) incorrect passwords get saved in the Docker volumes and do not change when you make a change to your .env file. To fix this, press Control+C in the terminal displaying Docker output, then `docker compose down`. Then open the Docker Desktop, click on volumes, and find and delete the volumes associated with the "ds6001databases" containers. Then return to the terminal,type `docker compose up`, and try again.
- If all else fails, you are far from the only one to hit a roadblock at this stage. Send me or one of the TAs a message and we can help.

[4 points]

In [76]:
```
# 1. Load needed environment variables
dotenv.load_dotenv()
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD')
MONGO_INITDB_ROOT_USERNAME = os.getenv('MONGO_INITDB_ROOT_USERNAME')
MONGO_INITDB_ROOT_PASSWORD = os.getenv('MONGO_INITDB_ROOT_PASSWORD')
MYSQL_ROOT_PASSWORD = os.getenv('MYSQL_ROOT_PASSWORD')

# 2. Test MySQL
dbserver = mysql.connector.connect(
    user='root',
    password=MYSQL_ROOT_PASSWORD,
    host='localhost',
```

```
    port='3306',
    #auth_plugin='mysql_native_password'
)

# 3. Test PostgreSQL
dbserver = psycopg.connect(
    user='postgres',
    password=POSTGRES_PASSWORD,
    host='localhost',
    port = '5432'
)
dbserver.autocommit = True

# 4. Test MongoDB
myclient = pymongo.MongoClient(f"mongodb://{MONGO_INITDB_ROOT_USERNAME}:{MON
myclient.list_databases()
```

Out[76]: <pymongo.synchronous.command_cursor.CommandCursor at 0x16cb51040>

Problem 2

For this problem, I created fake data that represents patients who saw a doctor and received one or more prescriptions. Your goal in this problem is to reorganize the data into a database schema that conforms to E. F. Codd's 3rd normal form, and to document the resulting relational database using an ER diagram.

Wrapping your head around the normal form rules and building and documenting a database schema can be very difficult. First, grant yourself a lot of grace and give yourself plenty of time to take on this problem. Mastering database schema and documentation are central skills that distinguish a data engineer from a data scientist or analyst, so it is worth taking time to think about this task.

The data are listed on the following Google sheet: https://docs.google.com/spreadsheets/d/11wUk5dg39lmI-_O0SQ0M5_5VGgzuGJKvSyMq4Tyxc9M/edit?usp=sharing. For this problem you will be manipulating the data. I only included 10 rows to enable you to use any method you like for working with the data, including point-and-click data manipulation on Google sheets or Excel. Or you can use `pandas` if you like. We are emphasizing the product over the process for now.

There are two tabs on this spreadsheet. The first tab contains the data and the second tab contains column descriptions and notes. These notes (some of which are unrealistic, but needed for this exercise) are:

- No two patients have the same name and date of birth, so `patient_name` and `date_of_birth` together uniquely identify a patient.

- No two drugs share the same brand name, so `prescribed_drug` uniquely identifies the drug.
- The data contain one row for every patient/prescription combination, and no patient receives more than one prescription for the same drug. So `patient_name`, `date_of_birth`, and `prescribed_drug` comprise a primary key in the original data.
- No two physicians in the data share the same name, so `prescribing_physician` uniquely identifies physicians.
- Patients do not change their insurance provider.
- Every physician works at exactly one hospital, no two hospitals share the same name, and every hospital exists at exactly one location.

Some things to keep in mind for this problem:

1. Remember that the first rule for 2nd normal form is that the data are in 1st normal form, and the first rule for 3rd normal form is that the data are already in 2nd normal form. So please focus on meeting the rules for 1st, then 2nd, then 3rd normal form *in that order*.
2. Some students have never worked with databases before, but some have. If you have seen databases before, you have some ideas about how database schema tend to look. But if you arrange the data to match an idea of how databases often look, you will more than likely over-engineer the database by creating too many tables or unnecessary columns. Instead, focus on the explicit rules for each normal form and stop once those rules are met.

In [77]:
```
url = "https://docs.google.com/spreadsheets/d/11wUk5dg39lmI-_O0SQ0M5_5VGgzuG

df = pd.read_csv(url)
df.head()
```

Out[77]:

	patient_name	date_of_birth	prescribed_drug	prior_conditions	patient_sex	patier
0	Nkemdilim Arendonk	2/21/1962	Amoxil	[Pneumonia, Diabetes, Rubella]	M	
1	Nkemdilim Arendonk	2/21/1962	Micronase	[Pneumonia, Diabetes, Rubella]	M	
2	Raniero Coumans	8/15/1990	Zosyn	[Appendicitis, Crohn's disease, Mononucleosis]	M	
3	Raniero Coumans	8/15/1990	Humira	[Appendicitis, Crohn's disease, Mononucleosis]	M	
4	Mizuki Debenham	3/12/1977	Inlyta	[Kidney Cancer]	F	Kaise

Part a

In words, address the following points regarding 1st normal form:

- Describe the problem or problems that exist in the original data that prevent it from meeting the requirements for 1st normal form.
- Describe a reorganization of the data that does conform to 1st normal form. If this schema has multiple tables, give each table a name that indicates what the rows in the table represent. List the columns that exist in each table, and if the number of rows in a table is different from the number of rows in the original data, explain why.

(It will help you to make these edits to the data now, but you won't show your final data until part d)

[8 points]

One problem I see is the prior_conditions column, it contains a list of conditions, violates the atomic value rule. To fix this, we could create a second table called conditions with three columns, patient_name, date_of_birth, and prior_condition, the primary key would be the combination of these three columns. The original table would keep its rows, but prior_conditions column would be removed. The new conditions table would have more rows because patients can have multiple prior conditions.

In [78]:
```
prescriptions = df.drop(columns=['prior_conditions'])
prescriptions.head(3)
```

Out[78]:

	patient_name	date_of_birth	prescribed_drug	patient_sex	patient_insurance	dru
0	Nkemdilim Arendonk	2/21/1962	Amoxil	M	Aetna	USA
1	Nkemdilim Arendonk	2/21/1962	Micronase	M	Aetna	
2	Raniero Coumans	8/15/1990	Zosyn	M	Cigna	Inte

In [79]:
```
conditions = df[['patient_name', 'date_of_birth', 'prior_conditions']].copy(
conditions['prior_conditions'] = conditions['prior_conditions'].str.strip('[
conditions = conditions.explode('prior_conditions').drop_duplicates()
conditions = conditions.rename(columns={'prior_conditions': 'prior_conditior

conditions.head(3)
```

Out[79]:

	patient_name	date_of_birth	prior_condition
0	Nkemdilim Arendonk	2/21/1962	Pneumonia
0	Nkemdilim Arendonk	2/21/1962	Diabetes
0	Nkemdilim Arendonk	2/21/1962	Rubella

Part b

In words, address the following points regarding 2nd normal form:

- Describe the problem or problems that exist in the 1st normal form data that prevent it from meeting the requirements for 2nd normal form.
- Describe a reorganization of the data that does conform to 2nd normal form. If this schema has multiple tables, give each table a name that connects to what the rows in the table represent. List the columns that exist in each table, and if the number of rows in a table is different from the number of rows in the original data, explain why.

(It will help you to make these edits to the data now, but you won't show your final data until part d)

[8 points]

By looking at the columns patient_sex, patient_insurance, drug_maker, and drug_cost, we can see that they do not depend on the entire primary key, violating the 2NF rule. To fix this, we can create two more tables. The first table, called patients, will contain patient_name, date_of_birth, patient_sex, and patient_insurance, with patient_name and

date_of_birth together as the primary key. The second table, called drugs, will contain prescribed_drug, drug_maker, and drug_cost, where prescribed_drug is the primary key.

The original prescriptions table will keep the same rows, but the 4 columns will be removed. The patients table will have fewer rows because each patient will appear only once. The drugs table will have one row for each unique prescribed drug.

In [80]:
```
patients = prescriptions[
    ['patient_name', 'date_of_birth', 'patient_sex', 'patient_insurance']
].drop_duplicates()

patients.head(3)
```

Out[80]:

	patient_name	date_of_birth	patient_sex	patient_insurance
0	Nkemdilim Arendonk	2/21/1962	M	Aetna
2	Raniero Coumans	8/15/1990	M	Cigna
4	Mizuki Debenham	3/12/1977	F	Kaiser Permanente

In [81]:
```
drugs = prescriptions[
    ['prescribed_drug', 'drug_maker', 'drug_cost']
].drop_duplicates()

drugs.head(3)
```

Out[81]:

	prescribed_drug	drug_maker	drug_cost
0	Amoxil	USAntibiotics	14.62
1	Micronase	Pfizer	20.55
2	Zosyn	Baxter International Inc	394.00

In [82]:
```
prescriptions = prescriptions.drop(
    columns=['patient_sex', 'patient_insurance', 'drug_maker', 'drug_cost']
)

prescriptions.head(3)
```

Out[82]:

	patient_name	date_of_birth	prescribed_drug	prescribing_physician	physician_me
0	Nkemdilim Arendonk	2/21/1962	Amoxil	Earnest Caro	University of (
1	Nkemdilim Arendonk	2/21/1962	Micronase	Earnest Caro	University of(
2	Raniero Coumans	8/15/1990	Zosyn	Pamela English	University of

Part c

In words, address the following points regarding 3rd normal form:

- Describe the problem or problems that exist in the 2nd normal form data that prevent it from meeting the requirements for 3rd normal form.
- Describe a reorganization of the data that does conform to 3rd normal form. If this schema has multiple tables, give each table a name that connects to what the rows in the table represent. List the columns that exist in each table, and if the number of rows in a table is different from the number of rows in the original data, explain why.

(It will help you to make these edits to the data now, but you won't show your final data until part d)

[8 points]

In [83]:
```
physicians = prescriptions[
    ['prescribing_physician', 'physician_medschool',
     'physician_years_experience', 'hospital']
].drop_duplicates()

physicians.head(3)
```

Out[83]:

	prescribing_physician	physician_medschool	physician_years_experience	hosp
0	Earnest Caro	University of California (Irvine)	14	UP Presbyter Shadys
1	Earnest Caro	University ofCalifornia (Irvine)	14	UP Presbyter Shadys
2	Pamela English	University of Michigan	29	Northwest Memc Hosp

In [84]:
```
hospitals = prescriptions[
    ['hospital', 'hospital_location']
].drop_duplicates()

hospitals.head(3)
```

Out[84]:

	hospital	hospital_location
0	UPMC Presbyterian Shadyside	Pittsburgh, PA
2	Northwestern Memorial Hospital	Chicago, IL
4	Houston Methodist Hospital	Houston, TX

In [85]:
```
prescriptions = prescriptions.drop(
    columns=['physician_medschool', 'physician_years_experience',
             'hospital', 'hospital_location']
)

prescriptions.head(3)
```

Out[85]:

	patient_name	date_of_birth	prescribed_drug	prescribing_physician
0	Nkemdilim Arendonk	2/21/1962	Amoxil	Earnest Caro
1	Nkemdilim Arendonk	2/21/1962	Micronase	Earnest Caro
2	Raniero Coumans	8/15/1990	Zosyn	Pamela English

The problem here is that columns such as physician_medschool and physician_years_experience depend on prescribing_physician, while hospital_location depends on hospital. To fix this, we will create two more tables, physicians and hospitals. The physicians table will contain prescribing_physician (PK), physician_medschool, physician_years_experience, and hospital. The hospitals table will contain hospital (PK) and hospital_location.

The original prescriptions table will keep the same rows, but all those columns will be removed. The physicians table will have one row for each unique physician, and the hospitals table will have one row for each unique hospital, so both will have fewer rows than the original table.

Part d

Display all of the tables in their entirety from the version of the data you designed for part (c) to meet the requirements of 3rd normal form.

You can use several different methods to display these tables. You can download the tables to CSV format then upload them into your notebook using `pd.read_csv()`, or you can represent them using Markdown table syntax using a tool such as https://www.tablesgenerator.com/markdown_tables, or you can include them as screenshot images if you want. (Please don't just provide external links however, as that will slow down our grading)

[4 points]

In [86]:
```
pd.set_option('display.max_rows', 5)
pd.set_option('display.max_columns', None)

display(prescriptions)
display(patients)
display(drugs)
display(conditions)
display(physicians)
```

```
display(hospitals)

pd.set_option('display.max_rows', None)
```

	patient_name	date_of_birth	prescribed_drug	prescribing_physician
0	Nkemdilim Arendonk	2/21/1962	Amoxil	Earnest Caro
1	Nkemdilim Arendonk	2/21/1962	Micronase	Earnest Caro
...	...	...	...	...
8	Bonnie Hooper	7/4/1951	Xeloda	Steven Garbutt
9	Bonnie Hooper	7/4/1951	Demerol	Steven Garbutt

10 rows × 4 columns

	patient_name	date_of_birth	patient_sex	patient_insurance
0	Nkemdilim Arendonk	2/21/1962	M	Aetna
2	Raniero Coumans	8/15/1990	M	Cigna
4	Mizuki Debenham	3/12/1977	F	Kaiser Permanente
5	Zoë De Witt	11/23/1947	F	Medicare
8	Bonnie Hooper	7/4/1951	F	Blue Cross Blue Shield

	prescribed_drug	drug_maker	drug_cost
0	Amoxil	USAntibiotics	14.62
1	Micronase	Pfizer	20.55
...	...	...	...
7	Demerol	Pfizer	37.50
8	Xeloda	Genentech	860.00

8 rows × 3 columns

	patient_name	date_of_birth	prior_condition
0	Nkemdilim Arendonk	2/21/1962	Pneumonia
0	Nkemdilim Arendonk	2/21/1962	Diabetes
...	...	...	...
8	Bonnie Hooper	7/4/1951	Pancreatic Cancer
8	Bonnie Hooper	7/4/1951	Sciatica

13 rows × 3 columns

	prescribing_physician	physician_medschool	physician_years_experience	hospita
0	Earnest Caro	University of California (Irvine)	14	UPM Presbyteria Shadysid
1	Earnest Caro	University ofCalifornia (Irvine)	14	UPM Presbyteria Shadysid
...	...	...	...	.
6	Hilde Ali	Colorado University Boulder	11	Mount Sina Hospita
8	Steven Garbutt	Ohio State University	36	UCS Medica Cente

8 rows × 4 columns

	hospital	hospital_location
0	UPMC Presbyterian Shadyside	Pittsburgh, PA
2	Northwestern Memorial Hospital	Chicago, IL
4	Houston Methodist Hospital	Houston, TX
5	Mount Sinai Hospital	New York, NY
8	UCSF Medical Center	San Francisco, CA

Problem 3

For this problem, you will be documenting and a building database that contains the entire collected works of Shakespeare.



The data were collected by Catherine Devlin for https://opensourceshakespeare.org/. The database will have five tables, and they will be brought into your Python environment as `pandas` dataframes in 3rd normal form by running this code:

In [87]:
```
repo = 'https://github.com/jkropko/DS-6001/raw/master/localdata/'
works = pd.read_csv(repo + 'Works.csv')
characters = pd.read_csv(repo + 'Characters.csv')
chapters = pd.read_csv(repo + 'Chapters.csv')
paragraphs = pd.read_csv(repo + 'Paragraphs.csv')

# convert column names to lowercase (needed for postgreSQL to work properly)
characters.columns = characters.columns.str.lower()
chapters.columns = chapters.columns.str.lower()
paragraphs.columns = paragraphs.columns.str.lower()
works.columns = works.columns.str.lower()

# works in the characters tables is a comma separated list.
# Break it out into multiple rows in a new table
charworks = characters[['charid', 'works']]
charworks.loc[:,'works'] = charworks['works'].str.split(',')
charworks = charworks.explode('works')
charworks = charworks.rename({'works':'workid'}, axis=1)
characters = characters.drop('works', axis=1)

#Remove empty rows
chapters = chapters.query("~chapterid.isnull()")
paragraphs = paragraphs.query("~paragraphid.isnull()")
charworks = charworks.query("~workid.isnull()")

# Add chapterid to paragraphs
paragraphs = pd.merge(paragraphs,
                      chapters.drop('description', axis=1),
                      how='inner',
                      on=['workid', 'section', 'chapter'])

#Remove unnecessary columns
paragraphs = paragraphs.drop(['paragraphtype', 'section', 'chapter'],
                             axis=1)
```

The five tables are:

works: One row per work authored by Shakespeare, with columns:

- `workid`: (primary key) a unique ID without spaces or special characters for the work
- `title`: the title, such as "Twelfth Night"
- `longtitle`: a longer title, if there is one, such as "Twelfth Night, Or What You Will"
- `date`: year of publication
- `genretype`: `t` is a tragedy, such as *Romeo and Juliet* and *Hamlet*; `c` is a comedy, such as *A Midsummer Night's Dream* and *As You Like It*; `h` is a history, such as *Henry V* and *Richard III*; `s` refers to Shakespeare's sonnets; `p` is a narrative (non-sonnet) poem, such as *Venus and Adonis* and *Passionate Pilgrim*
- `notes`: Column for notes from the database maintainer, currently all `NaN`

- `source` : whether the text was originally downloaded from the Moby Project lexicon or Project Gutenberg.
- `totalwords` : Total words in the work
- `totalparagraphs` : Total number of lines of dialogue for plays, or stanzas for poems

characters: One row per character that appears in at least one work by Shakespeare. Some characters, such as Antony or Henry IV, appear in multiple works. Columns:

- `charid` : (primary key) a unique ID for a character
- `charname` : character's name (some characters are different but have the same name, such as the First Musician in Othello and the First Musician in Romeo and Juliet). For poems, the character is "Poet"
- `abbrev` : an abbreviation of the character's name, if needed for reference to some other analyses
- `description` : a longer description of who the character is, if available
- `speechcount` : number of lines of dialogue delivered by the character throughout the works the character appears in

chapters: One row for every unique scene in a play, or for every distinct poem in a collection of poems. Columns:

- `workid` : a unique ID without spaces or special characters for the work
- `chapterid` : (primary key) a unique ID for the scene/poem
- `section` : the scene/poem number
- `chapter` : the act number, if available
- `description` : short description of where the scene takes place, for plays

paragraphs: One row for every line of dialogue that appears in a Shakespeare play, or for every distinct poem in a collection of poems. Columns:

- `workid` : a unique ID without spaces or special characters for the work
- `paragraphid` : (primary key) a unique ID for the line of dialogue/poem
- `paragraphnum` : the position of the paragraph within the ordered list of paragraphs within a chapter
- `charid` : the unique ID of the character delivering the line of dialogue/poem
- `plaintext` : the text of the dialogue/poem
- `phonetictext` : the text of the dialogue/poem in phonetic text, useful for training computers to generate audio of this spoken text
- `stemtext` : the stems of the words in the text, useful for text analyses such as sentiment analysis
- `charcount` : number of characters in the line
- `wordcount` : number of words in the line
- `chapterid` : unique ID for the scene/poem

charworks: One row for every unique combination of character and play. Most characters appear once, but some (such as Antony or Henry IV) appear multiple times. Columns

- `charid`: (primary key) unique ID for the character
- `workid`: (primary key) unique ID for the work

Part a

Please refer to the textbook's discussion of ER diagrams as you complete this problem.

For each of the following pairs of tables, write

1. Which column or columns the tables should be joined on
2. A description of your reasoning, in plain language, for whether each table matches to one or many rows in the other
3. Whether there is a one-to-one, one-to-many, many-to-one, or many-to-many relationship between the tables
4. Write the database markup language (DBML) code to represent this kind of a relationship.

If we are connecting two tables named `table_a` and `table_b` on a column named `joincolumn` in each table, then the DBML syntax is

- `Ref: table_a.joincolumn - table_b.joincolumn` if the relationship from `table_a` to `table_b` is one-to-one
- `Ref: table_a.joincolumn < table_b.joincolumn` if the relationship from `table_a` to `table_b` is one-to-many
- `Ref: table_a.joincolumn > table_b.joincolumn` if the relationship from `table_a` to `table_b` is many-to-one
- `Ref: table_a.joincolumn <> table_b.joincolumn` if the relationship from `table_a` to `table_b` is many-to-many

If there are multiple columns that must be joined on, provide the DBML code for each of these columns.

Part a, problem i

charworks and **works** [2 points]

Both tables can be joined using workid. One work can have many characters, but each row in charworks refers to only one work, many to one, relationship

```
Ref: charworks.workid > works.workid
```

Part a, problem ii

characters and **charworks** [2 points]

Joined by charid, one to many relationship. One character can appear in many works, but a row in charworks refers to only one character.

```
Ref: characters.charid < charworks.charid
```

Part a, problem iii

works and **paragraphs** [2 points]

Joined by workid, one to many relationship. One work can have many paragraphs, but a paragraph can belong to only one work.

```
Ref: works.workid < paragraphs.workid
```

Part a, problem iv

chapters and **works** [2 points]

Joined by workid, one to many relationship. A work can have many chapters, but a chapter can belong to only one work.

```
Ref: works.workid < chapters.workid
```

Part a, problem v

paragraphs and **chapters** [2 points]

Joined by chapterid and workid, many to one relationship. A chapter can have many paragraphs, but one paragraph can belong to only one chapter.

```
Ref: paragraphs.chapterid > chapters.chapterid
```

```
Ref: paragraphs.workid > chapters.workid
```

Part a, problem vi

characters and **paragraphs** (count groups that might say a line in unison, like Chorus or Witches, as one character) [2 points]

Joined by charid, one to many relationship. A character can appear in many paragraphs, but a paragraph can belong to only one character.

`Ref: characters.charid < paragraphs.charid`

Part b

Please find the "Using dbdiagram.io and dbdocs.io to Document Your Database" page on Canvas. It is available under Modules and "Getting Started with the Tools We'll Be Using". Read the page and watch the video about using dbdocs.io and dbdiagram.io to document a relational database with an ER diagram.

Use https://dbdiagram.io to create an ER diagram for the five Shakespeare data tables, then publish this diagram to a stable URL by pressing the "Publish to dbdocs" button. Paste a link here to your ER diagram on DBdocs.io. Then paste your DBML code from dbdiagrams.io into the box below.

A few notes:

You can use the `pandas_df_to_dbml()` function, defined below, to generate database markup language (DBML) code for each table that also lists each column's data type, then paste this code into dbdiagram.io (use `print()` around the output to see the correct formatting):

In [88]:
```
def pandas_df_to_dbml(df: pd.DataFrame, table_name: str) -> str:
    """
    Converts a pandas DataFrame to a DBML string.

    Args:
        df: The pandas DataFrame to convert.
        table_name: The name of the table in the DBML schema.

    Returns:
        A DBML string representing the DataFrame schema.
    """

    dbml_string = f"Table {table_name} {{\n"

    for column_name, column_type in df.dtypes.items():
        dbml_type = map_pandas_dtype_to_dbml_type(column_type)
        dbml_string += f"  {column_name} {dbml_type}\n"

    dbml_string += "}\n"
    return dbml_string

def map_pandas_dtype_to_dbml_type(dtype) -> str:
    """Maps a pandas dtype to a DBML type."""
    dtype_name = str(dtype)
    if "int" in dtype_name:
      return "int"
```

```
    if "float" in dtype_name:
      return "float"
    if "datetime" in dtype_name:
        return "datetime"
    return "varchar"
```

Use the syntax [pk] after a column name and data type to designate the columns that are primary keys in each table. In this case the primary keys are

Table	Primary Key column(s)
works	workid
characters	charid
paragraphs	paragraphid
chapters	chapterid
charworks	charid, workid

To draw the lines linking one table to another, include the `Ref:` syntax that you wrote for your answers to part (a).

[4 points]

```
Make sure you get the .ipynb file for this lab from the
module 6 page on Canvas

Then when you double-click this box, you'll see three
backticks before and after this text. Leave those alone

Type your code here, between the backticks


Table works {
  workid varchar [pk]
  title varchar
  longtitle varchar
  date int
  genretype varchar
  notes float
  source varchar
  totalwords int
  totalparagraphs int
}

Table characters {
  charid varchar [pk]
  charname varchar
  abbrev varchar
  description varchar
```

```
  speechcount float
}

Table chapters {
  workid varchar
  chapterid float [pk]
  section float
  chapter float
  description varchar
}

Table paragraphs {
  workid varchar
  paragraphid varchar [pk]
  paragraphnum varchar
  charid varchar
  plaintext varchar
  phonetictext varchar
  stemtext varchar
  charcount float
  wordcount float
  chapterid float
}

Table charworks {
  charid varchar
  workid varchar

  indexes {
    (charid, workid) [pk]
  }
}

Ref: charworks.workid > works.workid

Ref: charworks.charid > characters.charid

Ref: paragraphs.workid > works.workid

Ref: chapters.workid > works.workid

Ref: paragraphs.chapterid > chapters.chapterid

Ref: paragraphs.charid > characters.charid
```

link: https://dbdocs.io/jonathan-orellana/Shakespeare-ER-diagram

In [89]:
```
print(pandas_df_to_dbml(works, "works"))
print(pandas_df_to_dbml(characters, "characters"))
print(pandas_df_to_dbml(chapters, "chapters"))
```

```
print(pandas_df_to_dbml(paragraphs, "paragraphs"))
print(pandas_df_to_dbml(charworks, "charworks"))
```

```
Table works {
  workid varchar
  title varchar
  longtitle varchar
  date int
  genretype varchar
  notes float
  source varchar
  totalwords int
  totalparagraphs int
}

Table characters {
  charid varchar
  charname varchar
  abbrev varchar
  description varchar
  speechcount float
}

Table chapters {
  workid varchar
  chapterid float
  section float
  chapter float
  description varchar
}

Table paragraphs {
  workid varchar
  paragraphid varchar
  paragraphnum varchar
  charid varchar
  plaintext varchar
  phonetictext varchar
  stemtext varchar
  charcount float
  wordcount float
  chapterid float
}

Table charworks {
  charid varchar
  workid varchar
}
```

Problem 4

For this problem, you will use the five dataframes that comprise the Shakespeare database you used in problem 3 to initialize local databases using SQlite, MySQL, and

PostgreSQL.

All of the methods for initializing and connecting to databases in Python are listed in the textbook. Note that there are differences in code that depend on whether you are using SQlite, MySQL, or PostgreSQL.

For creating databases on the MySQL and PostgreSQL servers, you will connect directly to the database server and use the `.cursor()` method to interact with it. But once the database exists, please use the `pd.to_sql()` and `pd.read_sql_query()` methods from `pandas` and the `create_engine()` method from `sqlalchemy` (and not the `.cursor()` approach) to add data to or retrieve data from the database.

Before attempting this problem, please make sure your work for problem 1 runs without error, which will ensure your docker containers for MySQL and PostgreSQL are ready to use (SQlite does not require external software and can run without Docker).

Part a

Initialize a new Shakespeare database using SQlite via the `sqlite3` package. Next use the `pd.to_sql()` method to add the five Shakespeare dataframes to this database. Then, to prove that this worked, issue the following SQL query to the database which should display a dataframe listing all characters from Shakespeare's plays with more than 200 lines of dialogue (and the Poet too):

```
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

Finally, after running the query, use the `.commit()` and `.close()` methods on the Python variable containing the database connection to save your changes and prevent further changes.

[8 points]

In [90]:
```
db = sqlite3.connect("shakespeare.db")

works.to_sql("works", con=db, if_exists="replace", index=False)
characters.to_sql("characters", con=db, if_exists="replace", index=False)
chapters.to_sql("chapters", con=db, if_exists="replace", index=False)
paragraphs.to_sql("paragraphs", con=db, if_exists="replace", index=False)
charworks.to_sql("charworks", con=db, if_exists="replace", index=False)
```

Out[90]:
```
1346
```

In [91]:
```
query = """
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

```
"""

pd.read_sql_query(query, con=db)
```

Out[91]:

	charname	description	speechcount
0	Antony	(Marcus Antonius)	253.0
1	Cleopatra	queen of Egypt	204.0
2	Falstaff	Sir John Falstaff	471.0
3	Duke of Gloucester	brother to the King	285.0
4	Hamlet	son of the former king and nephew to the prese...	358.0
5	Henry V	Prince, King of England	377.0
6	Iago	Othello's ancient (?)	272.0
7	Othello	A noble Moor in the service of the Ventian state	274.0
8	Poet	the voice of Shakespeare's poetry	733.0
9	Richard III	son of Richard Plantagenet, duke of York; was ...	246.0
10	Rosalind	daughter to the banished Duke	201.0
11	Timon	None	210.0

In [92]:
```
db.commit()
db.close()
```

Part b

Use the `dotenv` package to import your MySQL database password into your Python environment (don't expose this password in your code). Then use this password to initialize a new Shakespeare database using MySQL and the `mysql.connector` package.

Next, use the `create_engine()` method from `sqlalchemy` and the `pd.to_sql()` method to add the five Shakespeare dataframes to this database. Then, to prove that this worked, use the `pd.read_sql_query()` method to issue the following SQL query to the database

```
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

Finally, after running the query, use the `.commit()` and `.close()` methods on the Python variable containing the database connection to save your changes and prevent further changes.

[8 points]

In [93]:
```
from dotenv import load_dotenv
from sqlalchemy.engine import URL

load_dotenv()

MYSQL_ROOT_PASSWORD = os.getenv("MYSQL_ROOT_PASSWORD")
```

In [94]:
```
dbserver = mysql.connector.connect(
    user="root",
    password=MYSQL_ROOT_PASSWORD,
    host="localhost",
    port=3306
)

cursor = dbserver.cursor()

try:
    cursor.execute("CREATE DATABASE shakespeare")
except mysql.connector.Error as e:
    if e.errno == 1007:
        cursor.execute("DROP DATABASE shakespeare")
        cursor.execute("CREATE DATABASE shakespeare")
    else:
        raise
```

In [95]:
```
url = URL.create(
    "mysql+mysqlconnector",
    username="root",
    password=MYSQL_ROOT_PASSWORD,
    host="localhost",
    port=3306,
    database="shakespeare"
)

engine = create_engine(url)

works.to_sql("works", con=engine, if_exists="replace", index=False)
characters.to_sql("characters", con=engine, if_exists="replace", index=False
chapters.to_sql("chapters", con=engine, if_exists="replace", index=False)
paragraphs.to_sql("paragraphs", con=engine, if_exists="replace", index=False
charworks.to_sql("charworks", con=engine, if_exists="replace", index=False)
```

Out[95]: 1346

In [96]:
```
query = """
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
"""

pd.read_sql_query(query, con=engine)
```

Out[96]:

	charname	description	speechcount
0	Antony	(Marcus Antonius)	253.0
1	Cleopatra	queen of Egypt	204.0
2	Falstaff	Sir John Falstaff	471.0
3	Duke of Gloucester	brother to the King	285.0
4	Hamlet	son of the former king and nephew to the prese...	358.0
5	Henry V	Prince, King of England	377.0
6	Iago	Othello's ancient (?)	272.0
7	Othello	A noble Moor in the service of the Ventian state	274.0
8	Poet	the voice of Shakespeare's poetry	733.0
9	Richard III	son of Richard Plantagenet, duke of York; was ...	246.0
10	Rosalind	daughter to the banished Duke	201.0
11	Timon	None	210.0

In [97]:
```
dbserver.commit()
cursor.close()
dbserver.close()
engine.dispose()
```

Part c

Use the `dotenv` package to import your PostgreSQL database password into your Python environment (don't expose this password in your code). Then use this password to initialize a new Shakespeare database using PostgreSQL and the `psycopg` package. (For PostgreSQL, you will need to set the `.autocommit` attribute of Python variable that connects to the PostgreSQL server to `True`, or you will receive cryptic errors.)

Next, use the `create_engine()` method from `sqlalchemy` and the `pd.to_sql()` method to add the five Shakespeare dataframes to this database (don't worry if a negative number is displayed. This is a known issue with PostgreSQL in Python but is not indicative of a problem). Then, to prove that this worked, use the `pd.read_sql_query()` method to issue the following SQL query to the database

```
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

Finally, after running the query, use the `.commit()` and `.close()` methods on the Python variable containing the database connection to save your changes and prevent further changes.

(If you see an error like `ObjectInUse: database "shakespeare" is being accessed by other users DETAIL:  There is 1 other session using the database.`, go to the terminal, type Control+C, then `docker compose down`, then `docker compose up`, then try again.)

[8 points]

In [98]:
```
load_dotenv()

POSTGRES_PASSWORD = os.getenv("POSTGRES_PASSWORD")
```

In [99]:
```
dbserver = psycopg.connect(
    user="postgres",
    password=POSTGRES_PASSWORD,
    host="localhost",
    port=5432,
    dbname="postgres"
)

dbserver.autocommit = True

cursor = dbserver.cursor()

cursor.execute("DROP DATABASE IF EXISTS shakespeare")
cursor.execute("CREATE DATABASE shakespeare")
```

Out[99]:
```
<psycopg.Cursor [COMMAND_OK] [IDLE] (host=localhost database=postgres) at 0
x16cb3ea50>
```

In [100…
```
url = URL.create(
    "postgresql+psycopg",
    username="postgres",
    password=POSTGRES_PASSWORD,
    host="localhost",
    port=5432,
    database="shakespeare"
)

engine = create_engine(url)

works.to_sql("works", con=engine, if_exists="replace", index=False)
characters.to_sql("characters", con=engine, if_exists="replace", index=False
chapters.to_sql("chapters", con=engine, if_exists="replace", index=False)
paragraphs.to_sql("paragraphs", con=engine, if_exists="replace", index=False
charworks.to_sql("charworks", con=engine, if_exists="replace", index=False)
```

Out[100…
```
-1
```

In [101…
```
query = """
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
"""
```

```
pd.read_sql_query(query, con=engine)
```

Out[101…

	charname	description	speechcount
0	Antony	(Marcus Antonius)	253.0
1	Cleopatra	queen of Egypt	204.0
2	Falstaff	Sir John Falstaff	471.0
3	Duke of Gloucester	brother to the King	285.0
4	Hamlet	son of the former king and nephew to the prese...	358.0
5	Henry V	Prince, King of England	377.0
6	Iago	Othello's ancient (?)	272.0
7	Othello	A noble Moor in the service of the Ventian state	274.0
8	Poet	the voice of Shakespeare's poetry	733.0
9	Richard III	son of Richard Plantagenet, duke of York; was ...	246.0
10	Rosalind	daughter to the banished Duke	201.0
11	Timon	None	210.0

In [102…

```
dbserver.commit()
cursor.close()
dbserver.close()
engine.dispose()
```

Problem 5

Colin Mitchell is a web-developer and artist who has a bunch of cool projects that play with what data can do on the internet. One of his projects is Today in History, which provides an API to access all the Wikipedia pages for historical events that happened on this day in JSON format. The records in this JSON are stored in the `['data']['events']` path. Here's the first listing for today (the day I ran this code):

In [103…

```
history = requests.get("https://history.muffinlabs.com/date")
history_json = json.loads(history.text)
events = history_json['data']['Events']
events[0]
```

Out[103…
```
{'year': '202 BC',
 'text': 'Second Punic War: At the Battle of Zama, Roman legions under Scip
io Africanus defeat Hannibal Barca, leader of the army defending Carthage.
[citation needed]',
 'html': '202 BC – 202 BC – <a href="https://wikipedia.org/wiki/Second_Puni
c_War" title="Second Punic War">Second Punic War</a>: At the <a href="http
s://wikipedia.org/wiki/Battle_of_Zama" title="Battle of Zama">Battle of Zam
a</a>, Roman legions under Scipio Africanus defeat Hannibal Barca, leader o
f the army defending Carthage.',
 'no_year_html': '202 BC – <a href="https://wikipedia.org/wiki/Second_Punic
_War" title="Second Punic War">Second Punic War</a>: At the <a href="http
s://wikipedia.org/wiki/Battle_of_Zama" title="Battle of Zama">Battle of Zam
a</a>, Roman legions under Scipio Africanus defeat Hannibal Barca, leader o
f the army defending Carthage.',
 'links': [{'title': 'Second Punic War',
   'link': 'https://wikipedia.org/wiki/Second_Punic_War'},
  {'title': 'Battle of Zama',
   'link': 'https://wikipedia.org/wiki/Battle_of_Zama'}]}
```

Here's the count of total events for the day I ran this code:

In [104…
```
len(events)
```

Out[104… 56

For this problem, you will use MongoDB and the `pymongo` package to create a local document store NoSQL database containing these historical events.

Part a

First, check that your work for problem 1 is still running without error. If so, then you have a MongoDB server running inside a Docker container.

Use `pymongo` to connect to the local MongoDB client, create a database named "history" and a collection within that database named "today". Because you will probably be running this code several times as you work, debugging as you go along, it's useful write code that deletes any existing "today" collections before creating a new "today" collection.

[4 points]

In [105…
```
load_dotenv()

MONGO_INITDB_ROOT_USERNAME = os.getenv("MONGO_INITDB_ROOT_USERNAME")
MONGO_INITDB_ROOT_PASSWORD = os.getenv("MONGO_INITDB_ROOT_PASSWORD")

client = pymongo.MongoClient(
    host="localhost",
    port=27017,
    username=MONGO_INITDB_ROOT_USERNAME,
    password=MONGO_INITDB_ROOT_PASSWORD,
```

```
    authSource="admin"
)

db = client["history"]

db.drop_collection("today")

collection = db.create_collection("today")
```

Part b

Insert all of the records in `events` into this collection. [4 points]

In [106…
```
collection.insert_many(events)
collection.count_documents({})
```

Out[106… 56

Part c

Issue the following query to find all of the records whose text contain the word "Virginia":

```
query = {
    "text":{
        "$regex": 'England'
    }
}
```

If there are no results that contain the word "England", choose a different word like "China" or "war". Display the count of the number of documents that match this query, display the output of the query, and generate a JSON formatted variable containing the output. [4 points]

In [107…
```
query = {
    "text": {
        "$regex": "England"
    }
}

collection.count_documents(query)
```

Out[107… 2

In [108…
```
results = list(collection.find(query))
results
```

Out[108…

```
[{'_id': ObjectId('6ad6653120b3569137f322ab'),
  'year': '1453',
  'text': "Hundred Years' War: Three months after the Battle of Castillon,
England loses its last possessions in southern France.",
  'html': '1453 – <a href="https://wikipedia.org/wiki/Hundred_Years%27_War"
title="Hundred Years\' War">Hundred Years\' War</a>: Three months after the
<a href="https://wikipedia.org/wiki/Battle_of_Castillon" title="Battle of C
astillon">Battle of Castillon</a>, England loses its last possessions in so
uthern France.',
  'no_year_html': '<a href="https://wikipedia.org/wiki/Hundred_Years%27_Wa
r" title="Hundred Years\' War">Hundred Years\' War</a>: Three months after
the <a href="https://wikipedia.org/wiki/Battle_of_Castillon" title="Battle
of Castillon">Battle of Castillon</a>, England loses its last possessions i
n southern France.',
  'links': [{'title': "Hundred Years' War",
    'link': 'https://wikipedia.org/wiki/Hundred_Years%27_War'},
   {'title': 'Battle of Castillon',
    'link': 'https://wikipedia.org/wiki/Battle_of_Castillon'}]},
 {'_id': ObjectId('6ad6653120b3569137f322d6'),
  'year': '1989',
  'text': 'The convictions of the Guildford Four are quashed by the Court o
f Appeal of England and Wales, after they had spent 15 years in prison.',
  'html': '1989 – The convictions of the <a href="https://wikipedia.org/wik
i/Guildford_Four" class="mw-redirect" title="Guildford Four">Guildford Four
</a> are quashed by the Court of Appeal of England and Wales, after they ha
d spent 15 years in prison.',
  'no_year_html': 'The convictions of the <a href="https://wikipedia.org/wi
ki/Guildford_Four" class="mw-redirect" title="Guildford Four">Guildford Fou
r</a> are quashed by the Court of Appeal of England and Wales, after they h
ad spent 15 years in prison.',
  'links': [{'title': 'Guildford Four',
    'link': 'https://wikipedia.org/wiki/Guildford_Four'}]}]
```


In [109…
```
from bson.json_util import dumps

results_json = dumps(results)
print(results_json)
```

[{"_id": {"$oid": "6ad6653120b3569137f322ab"}, "year": "1453", "text": "Hundred Years' War: Three months after the Battle of Castillon, England loses its last possessions in southern France.", "html": "1453 – Hundred Years' War: Three months after the Battle of Castillon, England loses its last possessions in southern France.", "no_year_html": "Hundred Years' War: Three months after the Battle of Castillon, England loses its last possessions in southern France.", "links": [{"title": "Hundred Years' War", "link": "https://wikipedia.org/wiki/Hundred_Years%27_War"}, {"title": "Battle of Castillon", "link": "https://wikipedia.org/wiki/Battle_of_Castillon"}]}, {"_id": {"$oid": "6ad6653120b3569137f322d6"}, "year": "1989", "text": "The convictions of the Guildford Four are quashed by the Court of Appeal of England and Wales, after they had spent 15 years in prison.", "html": "1989 – The convictions of the Guildford Four are quashed by the Court of Appeal of England and Wales, after they had spent 15 years in prison.", "no_year_html": "The convictions of the Guildford Four are quashed by the Court of Appeal of England and Wales, after they had spent 15 years in prison.", "links": [{"title": "Guildford Four", "link": "https://wikipedia.org/wiki/Guildford_Four"}]}]

Problem 6

Once you are finished working with databases, clear up the space on your computer by going to the terminal that you used to launch the Docker containers, press CONTROL + C on your keyboard to stop the containers, then type `docker compose down` to disconnect the volumes and networks. It's a good idea to make a practice out of doing these steps when you finish working with databases.

This problem isn't graded, and no need to write anything. But please do this anyway.